

Project Brief — App de Análise e Preparação de Ordens para IBKR

Para: o colega que vai desenvolver isto comigo **Objetivo deste documento:** dar-te o contexto completo — não só o *que* decidimos, mas *como* lá chegámos, para entrares no projeto com a cabeça no sítio certo. Há um segundo ficheiro (o prompt para o Claude Code, em inglês) que é a especificação técnica detalhada de construção. Este explica o raciocínio; esse diz o que construir.

1. Conclusão em uma frase

Vamos construir uma **aplicação local, só para nós**, que se liga à **conta IBKR através da API oficial**, faz análise técnica e backtesting, e **prepara ordens que nós confirmamos uma a uma** antes de serem enviadas. Arranca sempre em **paper trading**. Inclui um exportador das operações do ano em EUR para o IRS.

A app **informa e prepara**. Quem carrega no gatilho somos sempre nós.

2. O percurso até aqui (as decisões e porquê)

2.1. Começámos com a ideia errada — e corrigimos

A ideia inicial era a app **automatizar o site da corretora** (cliques no website) para dar ordens. Abandonámos isto, por boas razões: viola tipicamente os Termos de Serviço da corretora e pode levar a **bloqueio da conta**, é frágil (parte ao mínimo que a interface muda), e implica contornar autenticação. A leitura correta: quando uma corretora não expõe API, isso significa "não automatizar", não "automatizar à força pela interface".

Pivot: usar a **API oficial** da corretora.

2.2. A distinção que destrancou tudo

Não vamos *desenvolver a API* — ela já existe, é da corretora. Vamos desenvolver o **cliente** dessa API: o nosso programa que se liga à porta oficial que a corretora abriu de propósito para programas. Eles fazem a porta; nós fazemos quem bate à porta. Não estamos a forçar nada.

2.3. A fronteira de segurança que fixámos (importante para ti)

Esta é a espinha do desenho e **não é negociável**:

- O sistema analisa, gera sinais e **prepara** a ordem — mas a ordem só é transmitida quando o utilizador confirma **aquela ordem específica**, com todos os detalhes à frente.
- **Não existe** "confirmar tudo" em lote, **não existe** modo auto-executar, **não existe** loop autónomo que negocieie sem confirmação humana por ordem.
- **Paper trading por defeito.** Passar a live é um passo deliberado e separado, com fricção extra.
- **Kill switch** sempre acessível (cancela tudo e bloqueia novas ordens).
- **Limites de risco** aplicados no servidor *antes* de uma ordem poder sequer ser apresentada.
- **As credenciais nunca passam pelo código.** A autenticação é o login do utilizador no Gateway.
- **Registo de auditoria** de cada sinal, ordem, confirmação e fill.

Isto não é uma limitação técnica nossa — é uma decisão de desenho deliberada. Se o Claude Code (ou nós) for tentado a construir a versão "robô que negocia sozinho", a resposta é não. A app não é, e não pode tornar-se, um trader autónomo.

2.4. Escolha da corretora — dois requisitos em tensão

Tínhamos dois objetivos que puxam para corretoras diferentes:

- **API forte para automação** → aponta para a **Interactive Brokers (IBKR)**, que tem a API mais completa e madura para Python, ou a Alpaca (mais simples, mas orientada aos EUA e com acesso europeu limitado para conta real).
- **Relatório de IRS fácil em Portugal** → aponta para a **XTB**, que é a única das corretoras populares com sucursal fiscal em Portugal: comunica as operações à Autoridade Tributária (aparecem pré-preenchidas no Portal das Finanças) e dá um relatório anual já convertido para EUR à data de cada transação. Mas a XTB **não tem API aberta** para o que queremos.

Resolução: vamos de **IBKR pela API**, e o IRS resolve-se à **parte** no fim do ano com uma ferramenta portuguesa (Tax-Wizard ou IRSimples, que pegam no extrato da IBKR e geram o Anexo G/J), complementada pelo **nosso próprio módulo de exportação em EUR**. Assim não sacrificamos nenhum dos dois requisitos — usamos a ferramenta certa para cada um.

2.5. A IBKR é segura? (due diligence feita)

- Fundada em **1978** (~50 anos), cotada na **NYSE** (capitalização ~80 mil M), *capital próprio 21 mil M*.
- Regulada por reguladores de topo (**SEC, FCA, ASIC**, entre outros).
- Ativos protegidos pela **SIPC** (até 500.000 USD, sublimite 250.000 em cash) + apólice excedentária na Lloyd's. Um não-residente nos EUA é tratado igual a um residente.

- **Distinção crítica:** esta proteção cobre a **falência da corretora**, **não** a perda de valor de mercado. O risco real é perdermos dinheiro nas decisões de trading — e no day trading esse risco é alto, independentemente de a corretora ser sólida. A app não elimina isso; o paper-first existe precisamente para testarmos sem arriscar.

2.6. Tipo de conta e custos

- A **Lite não está disponível na Europa** (só EUA/Singapura). Em Portugal vamos de **IBKR Pro** de qualquer forma — e a Pro é a que tem a API a sério.
 - **Sem taxa de inatividade, sem mínimo de conta** (a inatividade foi eliminada em 2023). Seguro de abrir e deixar em paper durante semanas.
 - Dentro da Pro, escolher **Tiered** (não Fixed): mínimo por ordem muito menor (~0,35 vs 1,00 USD), o que é melhor para o padrão day-trading de muitas ordens pequenas/médias. **Nota de API:** ordens via SmartRouting podem usar Tiered; só ordens "directed" (encaminhamento forçado) ficam presas ao Fixed — por isso usamos SmartRouting.
 - **Custos escondidos a vigiar:** conversão de moeda (manual ~0,002% vs automática ~0,03% — fazer manual quando justifica) e **subscrições de dados de mercado** acima do tier gratuito.
-

3. As decisões técnicas (a conclusão concreta)

3.1. Caminho de API

TWS API via IB Gateway (não a Web API, para já). A Web API (REST/OAuth) é mais leve e sem desktop, mas é menos completa, mais lenta, e a ligação OAuth direta sem gateway só existe para institucionais. A TWS API é a mais madura para retail em Python.

3.2. Biblioteca — ATENÇÃO a esta

Usar `ib_async` (org `ib-api-reloaded`, $\geq 2.1.0$). **NÃO usar `ib_insync`** — foi **arquivado em março de 2024** após o falecimento do autor e está sem manutenção. O `ib_async` é o fork mantido que o sucede. Este é o erro nº 1 que se comete se formos de memória ou de tutoriais antigos.

3.3. Stack

- **Backend:** Python 3.11+ · FastAPI + uvicorn · `ib_async` (isolado atrás de um adapter) · pandas + pandas-ta (indicadores) · **Backtrader** (backtesting, com integração IBKR nativa e modelo event-driven que espelha o live) · SQLite via SQLAlchemy · structlog (auditoria).

- **Frontend:** React + TypeScript + Vite · **TradingView lightweight-charts** (velas + indicadores).
- **Processo:** o backend liga-se ao **IB Gateway** (preferível ao TWS: mais leve, auto-restart, sobrevive a inatividade).

3.4. Armadilhas operacionais a desenhar para dentro desde o início

- **O Gateway tem de estar a correr e não é headless** — o utilizador introduz user/password numa janela de login (sem GUI não é suportado).
- **Reautenticação semanal:** com 2FA, é preciso reautenticar ~1x/semana (tipicamente domingo) ou a ligação cai. A app deve detetar e mostrar um banner, não morrer em silêncio.
- **Uma sessão de trading por username:** para correr paper e live em simultâneo precisamos de um **segundo username** IBKR.
- **Dados de mercado na API são "off-platform":** mesmo o que vês grátis em tempo real no TWS pode chegar **atrasado** pela API sem subscrição. Numa conta paper não-financiada só há dados atrasados de Nível 1 + históricos. A app tem de funcionar bem com dados atrasados (`reqMarketDataType`) com fallback real-time → delayed → frozen) e mostrar claramente o tipo de dados por símbolo.

3.5. Backtesting — porquê Backtrader

Backtrader é maduro, tem integração IBKR e permite usar a mesma lógica em backtest e live (encaixa no nosso objetivo). O desenvolvimento estagnou ~2021 mas funciona bem. Estruturar para poder juntar depois um módulo VectorBT (varrer milhares de parâmetros rápido) sem reescrever a estratégia. NautilusTrader fica anotado como o caminho "se isto crescer" (paridade backtest/live de nível produção, mas curva de aprendizagem grande).

4. Plano de construção em fases (cada fase corre e testa-se antes da seguinte)

1. **Scaffold** — repo, config/.env (git-ignored), logging, SQLite, esqueletos FastAPI + React, banner de modo.
2. **Adapter (paper)** — ligação, estado, conta, posições, históricos, fallback de dados atrasados, reconexão.
3. **Dados live + indicadores + gráficos** — streaming, IndicatorEngine, lightweight-charts.
4. **Estratégias + feed de sinais** — estratégias de referência, modelo de Signal (ainda não coloca ordens).
5. **Motor de risco + kill switch** — limites no servidor, kill switch.

6. **Preparação + confirmação de ordens (paper)** — o fluxo prepara→revê→confirma, uma ordem de cada vez, com auditoria.
 7. **Backtesting** — Backtrader + métricas + gráficos.
 8. **Exportação IRS** — XLSX/CSV em EUR.
 9. **Guard rails do live** — fricção da troca de modo + ecrã de reconhecimento (decidimos nós se/quando passar a live).
-

5. O que já existe

- Um **prompt completo para o Claude Code (Opus 4.8) em VS Code**, em inglês, com 12 secções e estas 9 fases, e com toda a fronteira de segurança embutida (paper por defeito, confirmação por ordem, kill switch, sem auto-execução, tratamento de credenciais, auditoria). É a especificação de construção — começamos por aí.

6. Próximos passos sugeridos

1. Abrir a conta **IBKR Pro / Tiered** (cada um a sua, é pessoal).
 2. Instalar e configurar o **IB Gateway** + ativar acesso à API nas definições (é onde a maioria tropeça).
 3. Correr o Claude Code com o prompt, começando pela **Fase 1 (scaffold)** e **Fase 3 (dados + indicadores)**, que já se testam sem a conta toda configurada.
 4. Decidir entre nós a política de limites de risco por defeito (notional máximo por ordem, perda diária máxima, etc.).
-

Nota final, para alinharmos: o ponto de paragem mais provável no início não é o código — é pôr o IB Gateway a falar com a API. Conta com isso e não desanimes; o README que vamos gerar cobre esse setup.